锁的膨胀过程

关于偏向锁

预备知识CAS

什么是CAS呢？compare and swap 比较和交换，在intel的CPU中，使用cmpxchg指令实现；在Java发展初期，java语言是不能够利用硬件提供的这些便利来提升系统的性能的。而随着java不断的发展,Java本地方法(JNI)的出现，使得java程序越过JVM直接调用本地方法提供了一种便捷的方式，因而java在并发的手段上也多了起来；

CAS 操作包含三个操作数 —— 内存位置（V）、预期原值（A）和新值(B)。 如果内存位置的值与预期原值相匹配，那么处理器会自动将该位置值更新为新值 。否则，处理器不做任何操作。无论哪种情况，它都会在 CAS 指令之前返回该 位置的值；整个过程都是不可打断的，所以CAS是一个原子操作；主要是利用CPU的CAS指令，同时借助JNI来完成Java的非阻塞算法；

CAS的练习

假设有如下接口

```
package com.enjoy.cas;

public interface Account {
    //查询
    Integer query();

    //获取
    void acquire(Integer i);
}
```

有api针对这个Account接口的实现类的query和acquire调用

```
package com.enjoy.cas;

import java.util.ArrayList;
import java.util.List;

/**
 * zl
 */
public class AccountTest {

    public static void main(String[] args) throws InterruptedException {
        //test(new AccountNomal(10000));
        //test(new AccountSync(10000));
        test(new AccountCas(10000));
    }

    /**
     * 传入一个具体的Account来测试
     * @param account
     */
```

```
    static void test(Account account) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        //定义500个线程 每个线程针对account取20
        for (int i = 0; i < 500; i++) {
            ts.add(new Thread(() -> {
                account.acquire(20);
            }));
        }
        //启动
        for (Thread t : ts) {
            t.start();
        }
        //等待他们全部指向完
        for (Thread t : ts) {
            t.join();
        }
        //查询还剩多少
        System.out.println(account.query());
    }
}
```

如何保证acqurie的操作是线程安全呢?

线程不安全的实现

```
package com.enjoy.cas;

/**
 * 没有锁没有CAS
 *
 */
public class AccountNomal implements Account {
    public AccountNomal(int balance){
        this.balance=balance;
    }
    private Integer balance;
    @Override
    public Integer query() {
        return this.balance;
    }

    @Override
    public void acquire(Integer i) {
        this.balance -= i;
    }
}
```

运行结果：大于或者等于0;

线程安全的加锁实现

```
package com.enjoy.cas;
```

```java
/**
 * sync
 * result :0
 *
 */
public class AccountSync implements Account {
    public AccountSync(int balance){
        this.balance=balance;
    }
    private Integer balance;
    @Override
    public Integer query() {
        return this.balance;
    }

    @Override
    public void acquire(Integer i) {
        synchronized (this) {
            this.balance -= i;
        }
    }
}
```

运行结果：始终未0

线程安全的无锁实现

```java
package com.enjoy.cas;

import java.util.concurrent.atomic.AtomicInteger;

/**
 * cas
 * result :0
 *
 */
public class AccountCas implements Account {
    public AccountCas(int balance){
        this.balance=new AtomicInteger(balance);
    }
    private AtomicInteger balance;
    @Override
    public Integer query() {
        return this.balance.get();
    }

    @Override
    public void acquire(Integer i) {

            while(true) {
            int prev = balance.get();
            int next = prev - i;
            if(balance.compareAndSet(prev, next)) {
                break;
            }
        }
```

```
    }
}
```

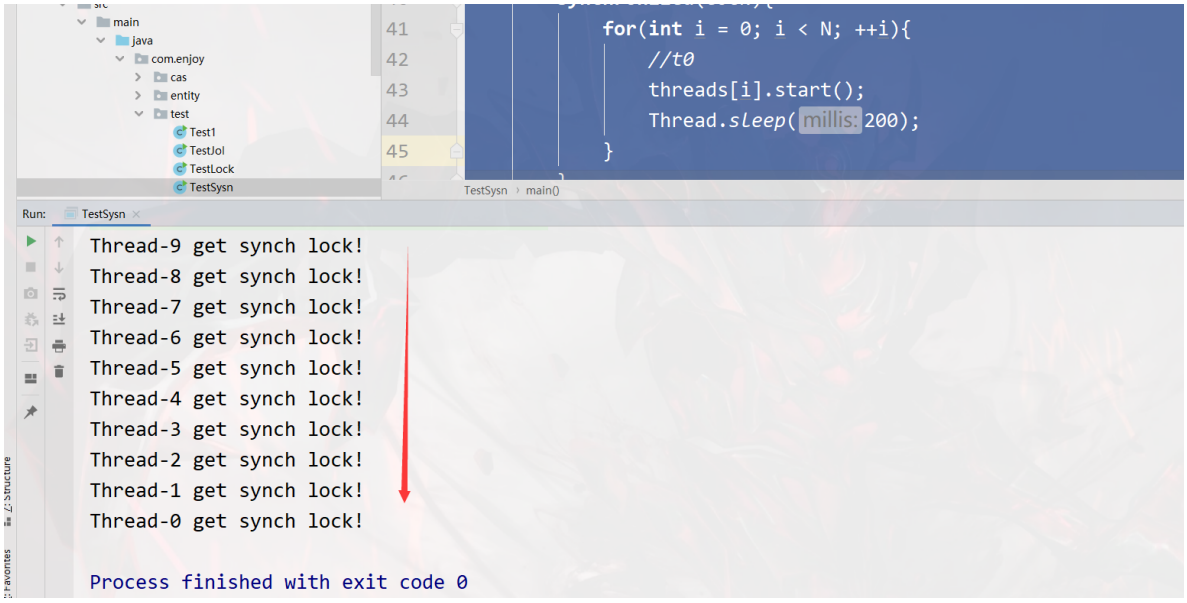
预备知识 公平锁非公平锁

考虑如下代码:

```
package com.enjoy.test;

/**
 * sync是否为公平锁
 */
public class TestSysn {
    //定义一把锁
    private static Object lock = new Object();

    public static void main(String[] args) throws InterruptedException {
        //线程的数量
        int N = 10;
        Thread[] threads = new Thread[N];


        for(int i = 0; i < N; ++i){
            threads[i] = new Thread(new Runnable(){
                public void run() {
                    /**
                     * 如果这里打印的结果是无序的则表示 非公平锁
                     * 有序则公平锁
                     * 结果是倒序 那么他就是公平锁吗?
                     * 其实很多时候可能我们对公平和非公平都理解错了
                     * 这里不管顺序打印还是倒序打印都只是对队列的特性验证
                     * 并不能说明他就是公平或者非公平
                     */
                    synchronized(lock){//t0 t1....t9 到一个队列当中的去阻塞
                        System.out.println(Thread.currentThread().getName() + "
get synch lock!");
                        try {
                            Thread.sleep(200);
                        } catch (InterruptedException e) {

                            e.printStackTrace();
                        }
                    }
                }

            });
        }

        //main 线程可以得到锁 持有了锁
        synchronized(lock){
            for(int i = 0; i < N; ++i){
                //t0
                threads[i].start();
                Thread.sleep(200);
            }
        }
```

```
//
//        for(int i = 0; i < N; ++i)
//            threads[i].join();


    }
}
```

结果分析：synchronized实现的锁，如果有多个线程阻塞，最先阻塞的最后执行；最后阻塞的最先执行

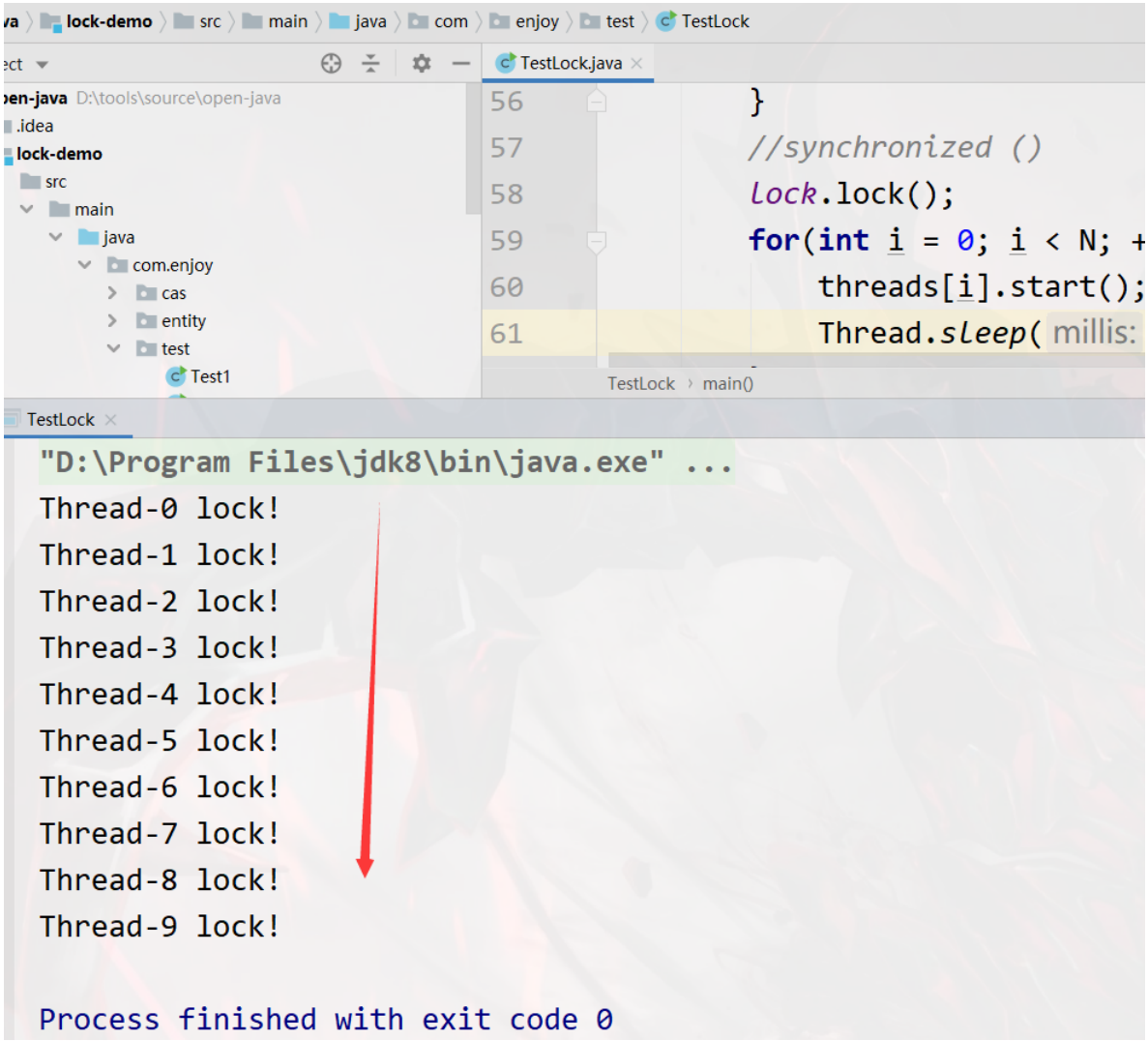


然后看看Lock实现的锁

```
package com.enjoy.test;

import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 * 这是一把公平锁
 * 公平锁非公平锁理解本身就错了
 *
 *
 * 非公平锁他们首先会在lock方法调用加锁的时候去抢锁（公平锁调用lock不会上来就拿锁）；
 * 如果加锁失败
 * 则去看为什么失败（是否锁被人持有了），他在判断的时候如果锁没有被人持有
 * 非公平锁有会去直接加锁（不会判断是否有人排队），成功则进入同步块；失败则进入队列
 * 进入队列后如果前面那个人是head则再次尝试加锁，成功则执行同步代码块，失败则park（真正的排队）
 *
 * 公平锁：第一次加锁的时候，他不会去尝试加锁，他会去看一下我前面有没有人排队，如果有人排队，我则进入
 * 队列（并不等于排队），然后还不死心，再次看一下我有没有拿锁的资格（前面那个人是否为head），
 * 如果有资格（前面那个人刚好为head）则继续拿锁，成功则执行同步块；失败则park（排队）
 *
 *
 * 一朝排队，永远排队
 *
```

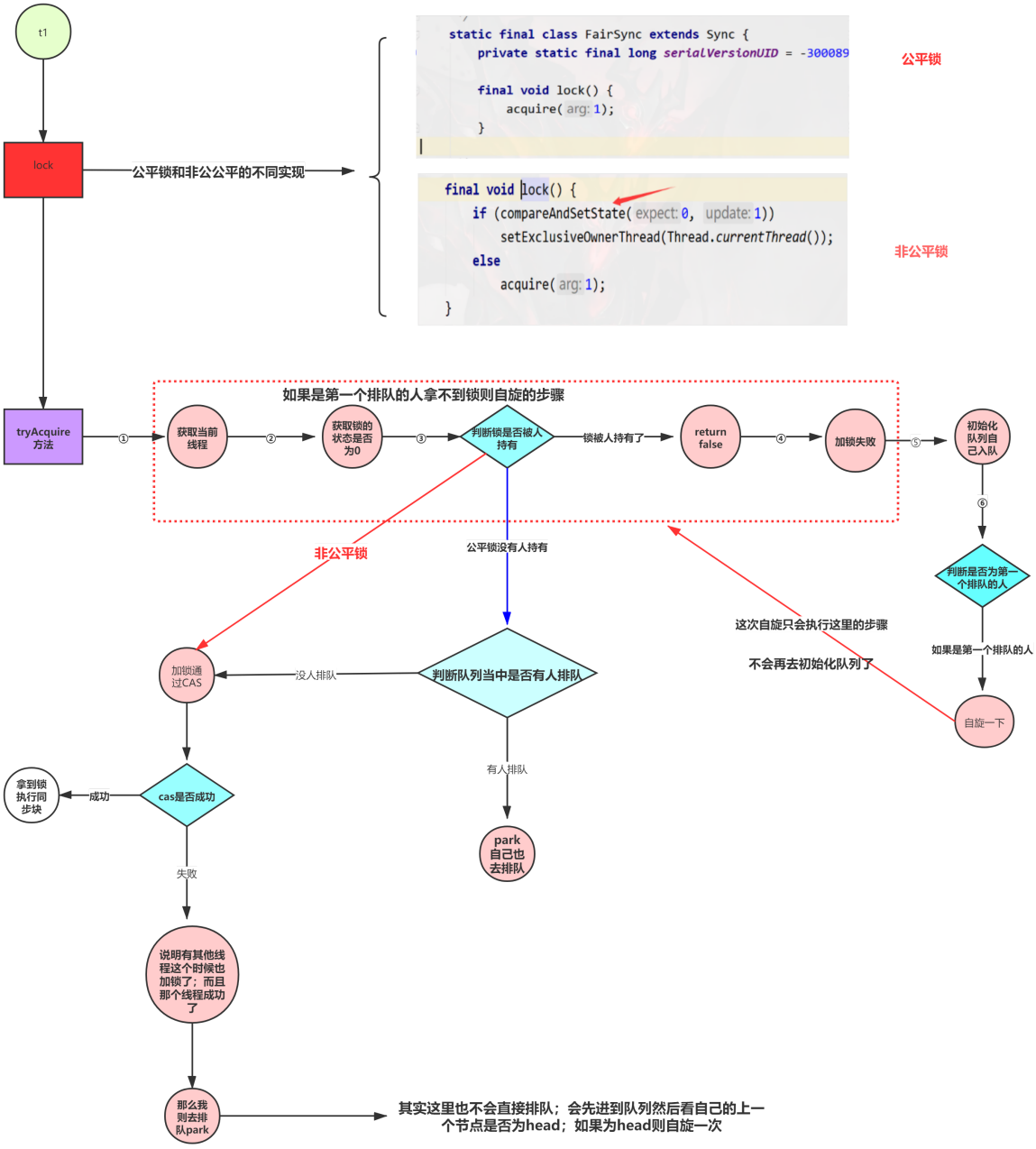
```
 */
public class TestLock {

    private static Lock lock = new ReentrantLock();

    public static void main(String[] args) throws InterruptedException {

        int N = 10;
        Thread[] threads = new Thread[N];
        for(int i = 0; i < N; ++i){
            threads[i] = new Thread(new Runnable(){
                public void run() {

                    /**
                     *
                     * 独占锁---顾名思义
                     * t1------t9全部在这里阻塞，并且进入到队列
                     * 等待持有锁的线程释放锁之后才从队列当中把这些阻塞的线程唤醒
                     *
                     */
                    lock.lock();
                    System.out.println(Thread.currentThread().getName() + "
lock!");
                    try {
                        Thread.sleep(20);
                    } catch (InterruptedException e) {
                        // TODO Auto-generated catch block
                        e.printStackTrace();
                    }
                    lock.unlock();
                }

            });
        }
        //synchronized ()
        lock.lock();
        for(int i = 0; i < N; ++i){
            threads[i].start();
            //Thread.sleep(200);
        }
       // lock.unlock();

        for(int i = 0; i < N; ++i)
            threads[i].join();
    }
}
```

结果分析：lock实现的同步如果有多个线程阻塞，唤醒的时候是按阻塞顺序唤醒的；关于公平锁和非公平锁可以参数我写的注释



关于公平锁和非公平锁的加锁流程和区别

好好看懂这幅图就懂了



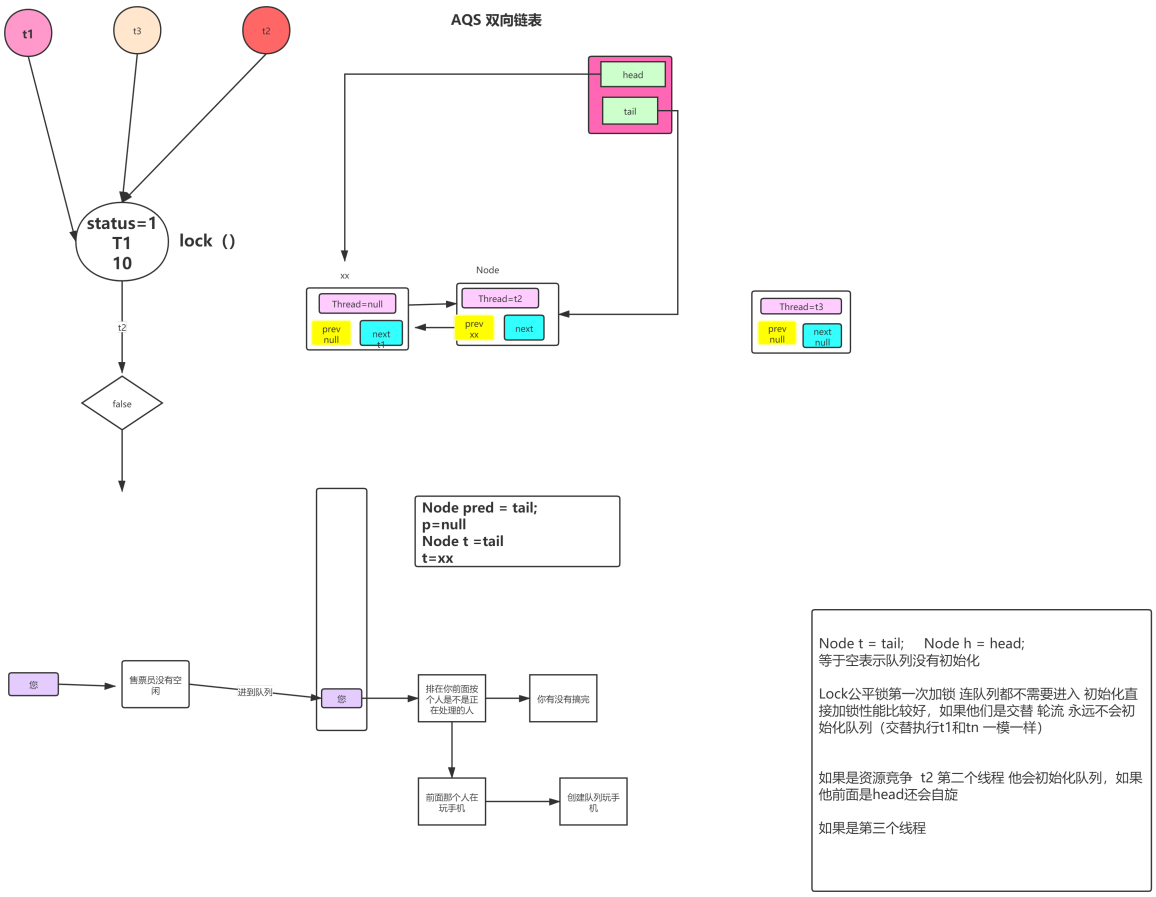
关于队列如何设计和形成

1、AQS类的设计主要属性

```
private transient volatile Node head; //队首

private transient volatile Node tail;//尾

private volatile int state;//锁状态，加锁成功则为1，重入+1 解锁则为0
```

2、Node类的设计

```
public class Node{
    volatile Node prev;
    volatile Node next;
    volatile Thread thread;
}
```



最后关于synchronized是否公平锁

其实前面那个小例子（倒序和顺序）不足以说明什么问题；synchronized是否公平锁得看线程在没有入队之前是否抢锁（C++代码）；入队之后就和公平不公平无关了；一朝排队；永远排队；

然后关于队列，关于双向链表好好理解一下就行